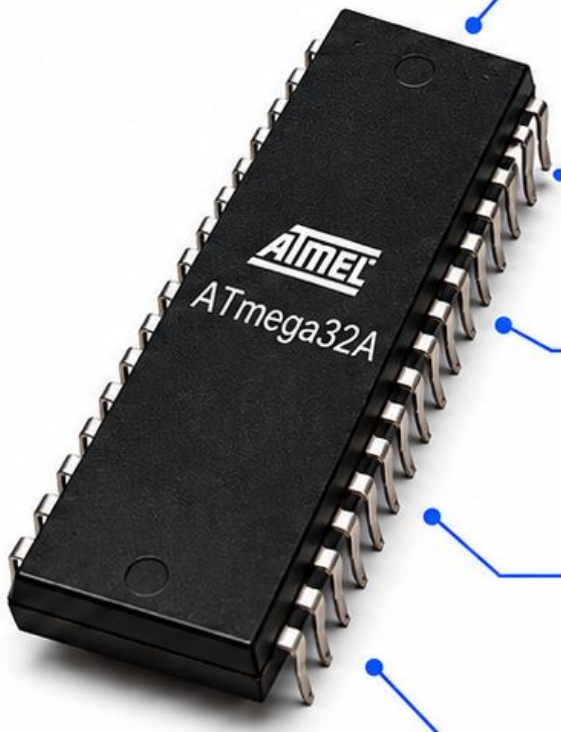
NTI National Telecommunication Institute

# ADC and DAC in Embedded Systems

## Analog signals, sampling, quantization, ADC types, and ATmega32 ADC registers

### Session Topics

- What are analog and digital signals?
- What are ADC and DAC?
- Frequency, crystal, and ADC clock
- Sampling and Nyquist theorem
- Quantization and quantization error
- Ramp ADC and SAR ADC
- ATmega32 ADC registers and example code




**10-bit ADC**
Resolves analog signals into 10-bit digital values.

**8 analog channels**
Multiplexed inputs: ADC0 to ADC7.

**Uses SAR ADC**
Successive Approximation Register architecture for fast and efficient conversion.

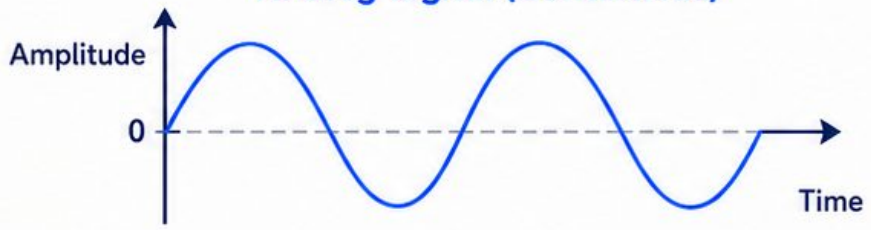
**Works with Vref and ADC clock**
Conversion depends on reference voltage and ADC clock frequency.

**Result stored in ADCL/ADCH**
10-bit result right-aligned across ADCL and ADCH registers.

NTI National Telecommunication Institute

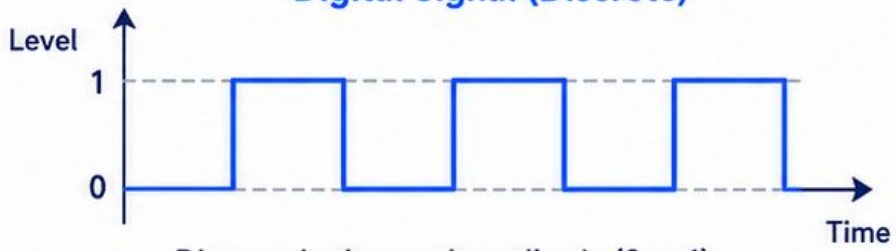
# Analog, Digital, ADC, and DAC

### Analog Signal (Continuous)




Continuous in time and amplitude

### Digital Signal (Discrete)




Discrete in time and amplitude (0 or 1)

- Analog signals are continuous in time and amplitude.
- Digital signals take on discrete values, typically 0 and 1.
- **ADC** (Analog-to-Digital Converter) converts an analog voltage to a digital number.
- **DAC** (Digital-to-Analog Converter) converts a digital number back to an analog voltage.

### Analog to Digital (ADC)


### Digital to Analog (DAC)

NTI National Telecommunication Institute

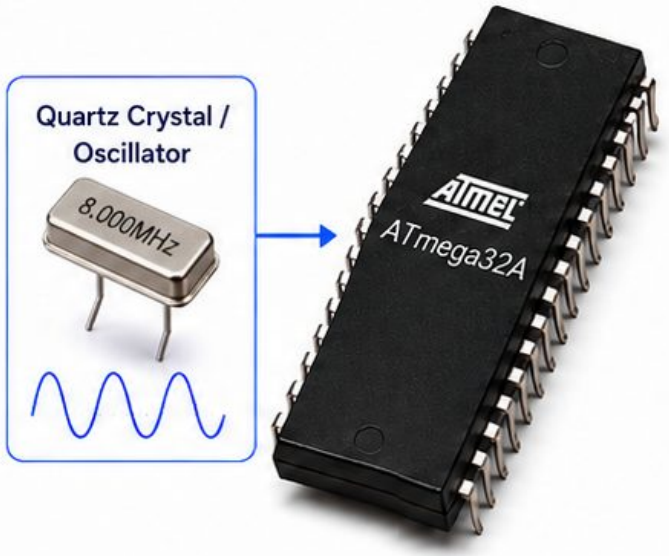
# Frequency, Crystal, and ADC Clock

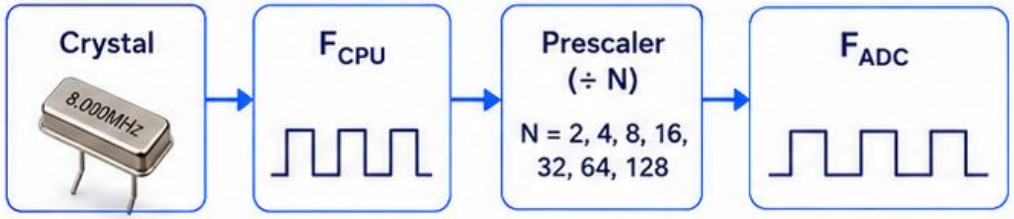
**Understanding the clock system in ATmega32A for accurate ADC conversions.**

### Crystal (Oscillator)
A quartz crystal provides a very stable clock source by resonating at a fixed frequency.

### MCU System Clock ($F_{CPU}$)
The crystal drives the MCU clock ($F_{CPU}$), which controls instruction execution and peripherals.

### ADC Clock ($F_{ADC}$)
The ADC needs a lower clock frequency for accurate and consistent 10-bit conversions.




### Crystal = Stable Clock
The crystal provides a precise and stable reference frequency.

### MCU Uses $F_{CPU}$
The MCU runs on $F_{CPU}$ for all operations and peripherals.

### ADC Needs Lower Clock
A lower ADC clock ($F_{ADC}$) ensures accurate conversions and reduces error.

### Recommended $F_{ADC}$ Range
For accurate 10-bit conversion: about 50 kHz to 200 kHz.

### Key Formulas
- $f = \frac{1}{T}$ where $f$ = frequency, T = time period
- $F_{ADC} = \frac{F_{CPU}}{\text{Prescaler}}$

### Example
If $F_{CPU}$ = 8 MHz and prescaler = 64,

then $F_{ADC} = \frac{8\ \text{MHz}}{64} = 125\ \text{kHz}$

### ADC Clock Recommendation
For accurate 10-bit conversion, keep $F_{ADC}$ in the range of about **50 kHz to 200 kHz.**

### Clock Visual
Sine Wave (Crystal)

Clock Pulses ($F_{CPU}$)

NTI National Telecommunication Institute

# Quantization Basics

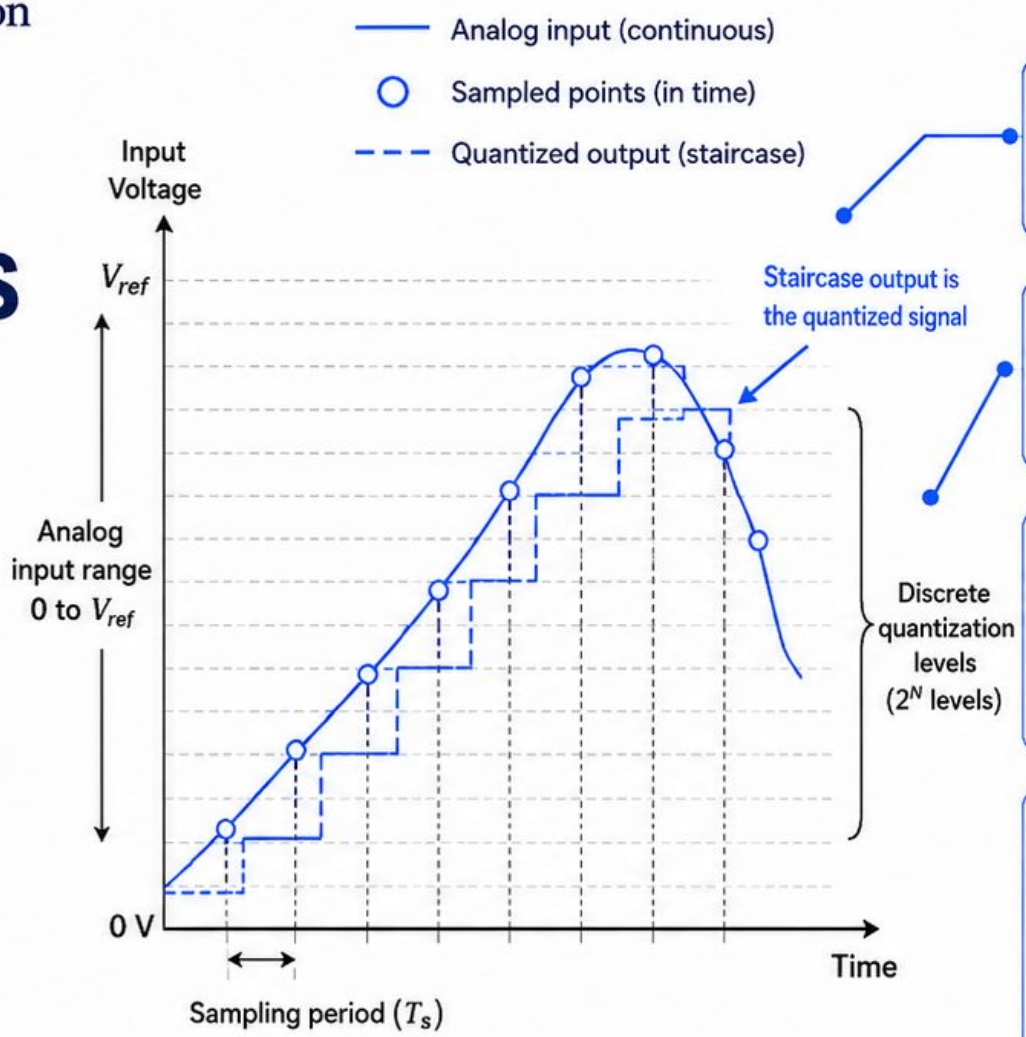
**Quantization maps each sampled value to the nearest value among a finite set of discrete levels.**

### Sampling gives time points
The continuous analog signal is sampled at regular intervals to get discrete-time values.

### Quantization maps to levels
Each sample is mapped to the nearest of the available digital levels.

### 10-bit ADC has $2^{10} = 1024$ levels
For an N-bit ADC, the number of quantization levels is $2^N$.

### Small quantization error
The difference between the actual value and the quantized value is the quantization error.




### Step size (LSB)
Step size (LSB) = $\frac{V_{ref}}{2^N}$

### Quantization error
Quantization error ≈ ±0.5 LSB

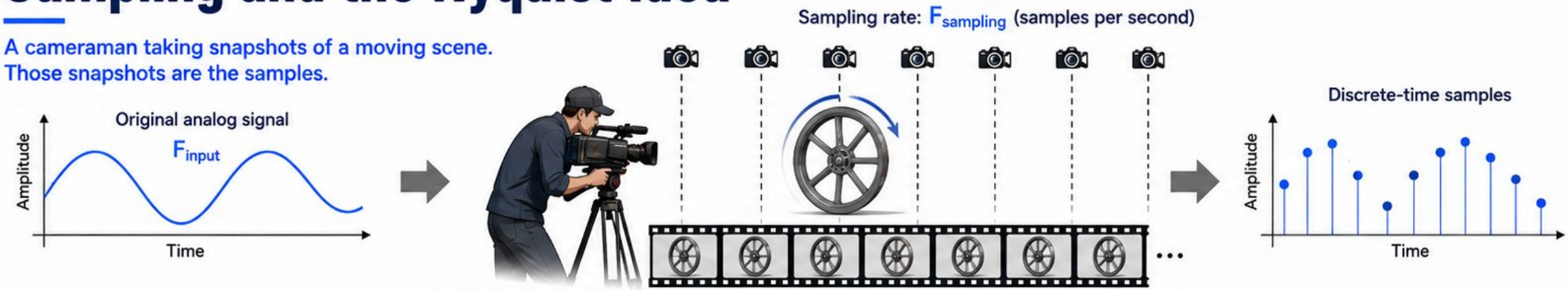
### Number of levels
N-bit ADC has $2^N$ levels.
For 10-bit: $2^{10} = 1024$ levels.

### Example
With $V_{ref} = 5$ V and N = 10 bits:
LSB = $\frac{5}{2^{10}}$ V = 4.88 mV
Quantization error ≈ ±2.44 mV

NTI National Telecommunication Institute

# Sampling and the Nyquist Idea

A cameraman taking snapshots of a moving scene.
Those snapshots are the samples.

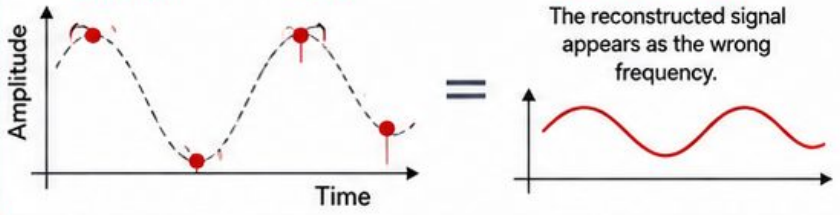



**Nyquist Idea (in one line)**

To reconstruct a signal correctly, $F_{sampling} >= 2 \times F_{input}$

## Undersampling (Too Few Samples)

$F_{sampling} < 2 \times F_{input}$

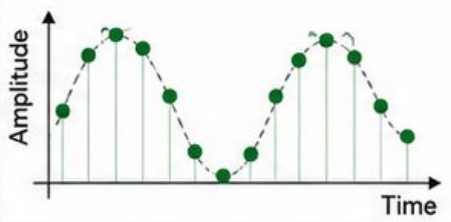



**Result: Aliasing**

The reconstructed signal appears as the wrong frequency.

## Proper Sampling (Enough Samples)

$F_{sampling} >= 2 \times F_{input}$




**Result: Accurate Reconstruction**

The original signal can be reconstructed correctly.

Higher sampling rate means more snapshots—more detail and a truer representation of the original scene.

**More samples, more truth.**

NTI National Telecommunication Institute

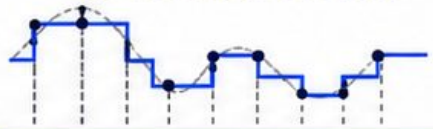
# The Quantization Process

**Converting a continuous analog signal into a digital representation step-by-step.**

**Analog input**
Continuous-time analog signal

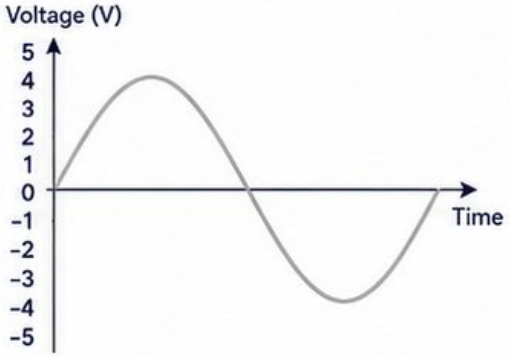
**Sampling**
Signal measured at discrete time instants

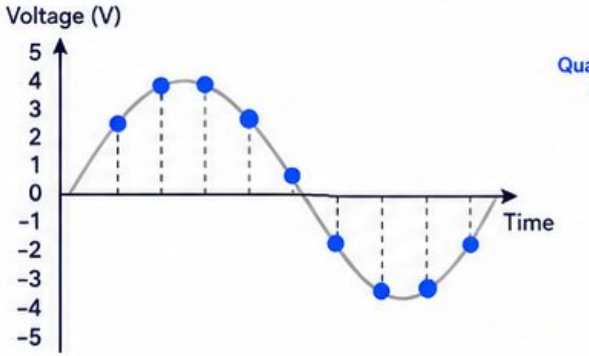
**Quantization**
Each sample is mapped to the nearest discrete level

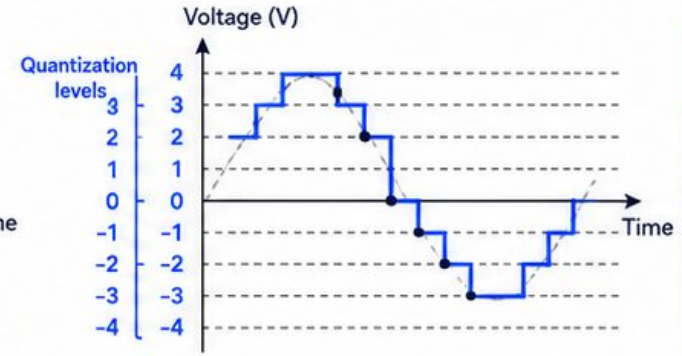


**Binary output**
Quantized levels encoded as binary numbers

000 001 010 011
100 101 110 111



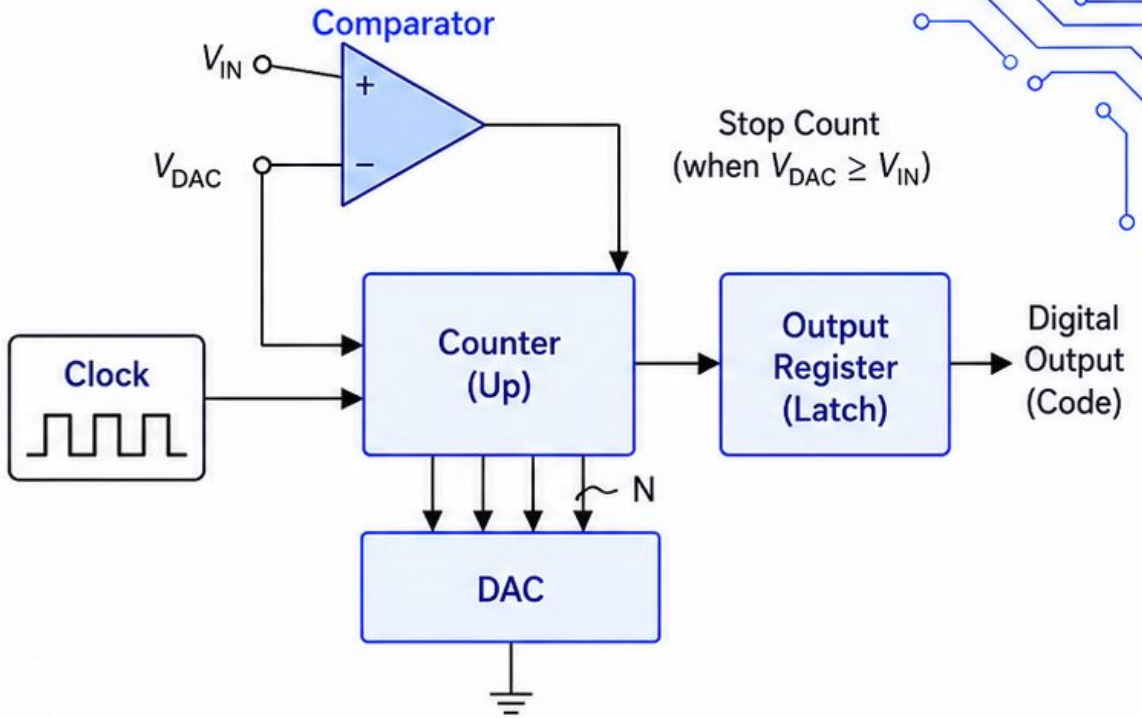




| Sample Voltage (V) | Nearest Level (V) | Digital Code (4-bit) |
|---|---|---|
| 0.8 V | 1 V | 0001 |
| 2.4 V | 2 V | 0010 |
| 4.1 V | 4 V | 0100 |

Quantization introduces a small **approximation error**—the difference between the actual sample value and the nearest quantization level.

NTI | National Telecommunication Institute

# Ramp ADC (Counter-Type ADC)




**A counter increments upward from 0. The DAC output ramps up in steps. When $V_{DAC} \approx V_{IN}$, the comparator stops the count. The counter value is the digital output.**

- Clock drives the counter.
- Counter output feeds the DAC and also the Output Register.
- Comparator compares $V_{DAC}$ to $V_{IN}$.
- When $V_{DAC} \geq V_{IN}$, the comparator stops the counter.
- The Output Register holds the final code.

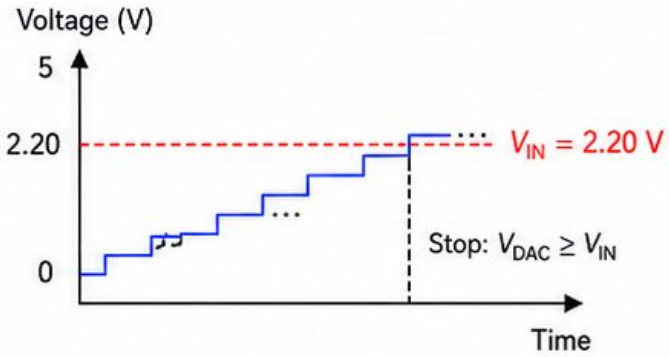
Simple architecture but relatively slow because it may count through many steps before reaching $V_{IN}$.

### Worked Example (4-bit, 0–5 V, $V_{IN}$ = 2.20 V)

- Resolution (LSB) = $\frac{5\text{ V}}{2^4}$ = 0.3125 V
- The DAC output steps: 0, 1, 2, …, 15 LSBs
- Code $0111_2 = 7_{10}$
- $V_{DAC} = 7 \times 0.3125 = 2.1875\text{ V} \approx 2.19\text{ V}$
- Since 2.19 V < 2.20 V and next code (8) gives 2.50 V > 2.20 V, the comparator stops at 0111.
- Output Code = $0111_2$ (7)

| Code (4-bit) | Decimal | $V_{DAC}$ (V) |
|---|---|---|
| 0110 | 6 | 1.875 |
| 0111 | 7 | 2.1875 ← Output |
| 1000 | 8 | 2.500 |

### How it works (ramp up and stop)



NTI | National Telecommunication Institute

# SAR ADC

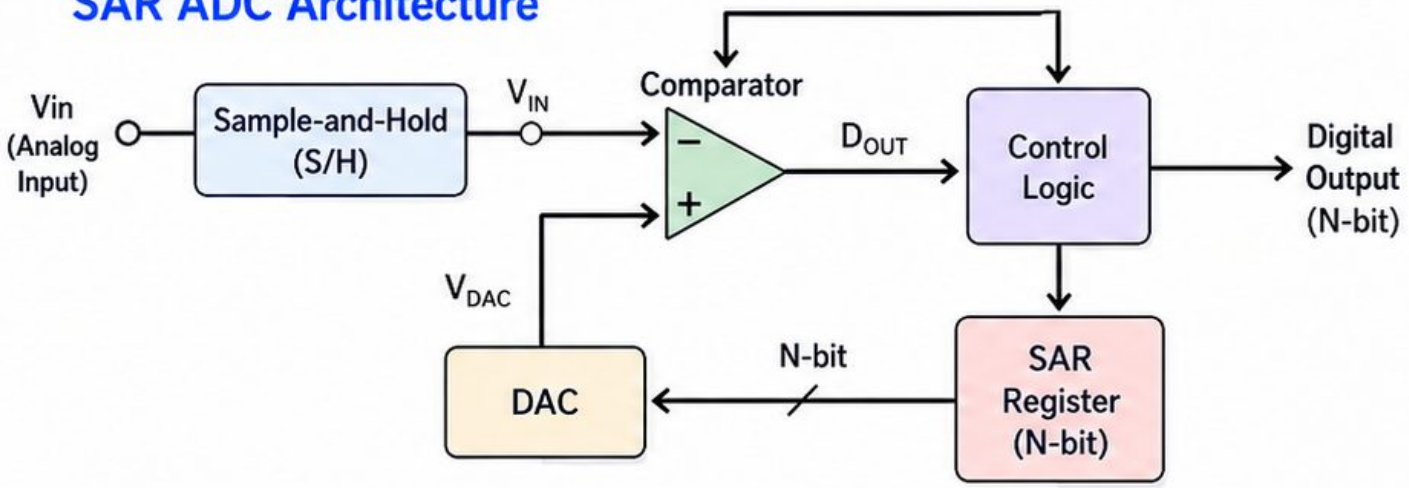
## (Successive Approximation ADC)

A SAR ADC converts an analog input to digital using a binary search. It tests bits starting from the MSB to the LSB and keeps the bit if the DAC output does not exceed the input.

- Tests bits from **MSB to LSB**.
- **Much faster** than ramp ADC (no counting needed).

**ATmega32 uses a SAR ADC.**

### SAR ADC Architecture




### Worked Example: 4-bit SAR ADC, 0–5 V, $V_{in} \approx 3.10$ V

| Step | Bit Trial (MSB → LSB) | DAC Output (V) | Compare with $V_{in}$ (≈ 3.10 V) | Decision | Register After Step |
|---|---|---|---|---|---|
| 1 | **1** 0 0 0 | 2.500 | 2.500 < 3.10 | Accept | 1 0 0 0 |
| 2 | 1 **1** 0 0 | 3.750 | 3.750 > 3.10 | Too High → Clear bit | 1 0 0 0 |
| 3 | 1 0 **1** 0 | 3.125 | 3.125 ≤ 3.10 | Accept | 1 0 1 0 |
| 4 | 1 0 1 **1** | 3.4375 | 3.4375 > 3.10 | Too High → Clear bit | 1 0 1 0 |

Final Output = $1010_2 = 10_{10}$ → $V_{out} = 10/16 \times 5\ V = 3.125\ V$ (about 3.125 V)

# ATmega32 ADC Registers

## ADMUX (0x27)

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | REFS1 | REFS0 | ADLAR | MUX4 | MUX3 | MUX2 | MUX1 | MUX0 |
| | Reference Select | | Left Adjust Result | Analog Channel Select (ADC0–ADC31) | | | | |

- **REFS1:0** – Reference select (00=AREF, 01=AVCC, 10=Internal 2.56V, 11=Reserved)
- **ADLAR** – Left adjust result in ADCH/ADCL
- **MUX4:0** – Select analog input channel

## ADCSRA (0x26)

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | ADEN | ADSC | ADATE | ADIF | ADIE | ADPS2 | ADPS1 | ADPS0 |
| | ADC Enable / Start / Auto Trigger | | | Interrupt Flags / Enable | | ADC Prescaler Select | | |

- **ADEN** – ADC Enable
- **ADSC** – ADC Start Conversion
- **ADATE** – Auto Trigger Enable
- **ADIF** – ADC Interrupt Flag
- **ADIE** – ADC Interrupt Enable
- **ADPS2:0** – ADC Clock Prescaler (2, 4, 8, 16, 32, 64, 128)

## ADCL (0x24)

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | ADC Data[7:0] | | | | | | | |

- ADC Data Low Byte
- Read ADCL first, then ADCH for correct 10-bit result

## ADCH (0x25)

| Bit | 7 | 6 | 5 | 4 | 3 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| | – | – | ADC Data[9:8] | | ADC Data[7:0] | | |

- ADC Data High Byte
- Bits 1:0 are valid when ADLAR = 0

## SFIOR (0x50)

| Bit | 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|---|
| | – | – | – | – | ADTS2 | ADTS1 | ADTS0 | – |
| | | | | | Auto Trigger Source Select | | | |

- **ADTS2:0** – Auto Trigger Source Select (000=Free Running, 001=Analog Comparator, 010=Ext Int0, 011=Timer/Counter0 Compare Match, 100=Timer/Counter0 Overflow, 101=Timer/Counter1 Compare Match B, 110=Timer/Counter1 Overflow, 111=Timer/Counter1 Capture Event)

## PORTA (ADC0–ADC7) Pins

| PA7 (ADC7) | PA6 (ADC6) | PA5 (ADC5) | PA4 (ADC4) | PA3 (ADC3) | PA2 (ADC2) | PA1 (ADC1) | PA0 (ADC0) |
|---|---|---|---|---|---|---|---|

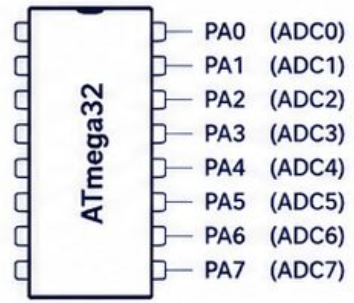
- Analog input channels ADC0 to ADC7
- Set corresponding PORTA pin as input
- Digital input buffer can be disabled via DIDR0 for better accuracy

**IMPORTANT NOTE**

The ATmega32 ADC is 10-bit. Result is in ADCH:ADCL.

For full precision, read **ADCL first**, then **ADCH**.

**ANALOG INPUT CHANNELS ADC0–ADC7**



# ATmega32 ADC Example Code

```
void ADC_Init(void){
   ADMUX   = (1<<REFS0);                              // AVcc with external capacitor at AREF
   ADCSRA  = (1<<ADEN) | (1<<ADPS2) | (1<<ADPS1);     // Enable ADC, prescaler = 64
}

uint16_t ADC_Read(uint8_t ch){
   ADMUX   = (ADMUX & 0xE0) | (ch & 0x07);            // Select channel (0–7)
   ADCSRA |= (1<<ADSC);                               // Start conversion
   while (ADCSRA & (1<<ADSC));                        // Wait until conversion complete
   return ADCW;                                       // Read 10-bit result (0–1023)
}

// Example usage
uint16_t value;
float voltage;
value = ADC_Read(0);                 // Read from ADC channel 0 (ADC0)
voltage = value * 5.0 / 1024.0;      // Convert to voltage (Vref = 5V)
```

**Select Vref and channel**
Set reference voltage (REFS0) and choose ADC channel (0-7).

**Enable ADC**
Set ADEN = 1 to turn on the ADC module.

**Set prescaler**
Choose ADC clock prescaler (e.g., 64) for proper ADC conversion speed.

**Start conversion**
Set ADSC = 1 to start a single conversion.

**Wait until complete**
ADSC is cleared by hardware when the conversion finishes.

**Read result**
Read ADCW (10-bit result, 0–1023) and use as needed.